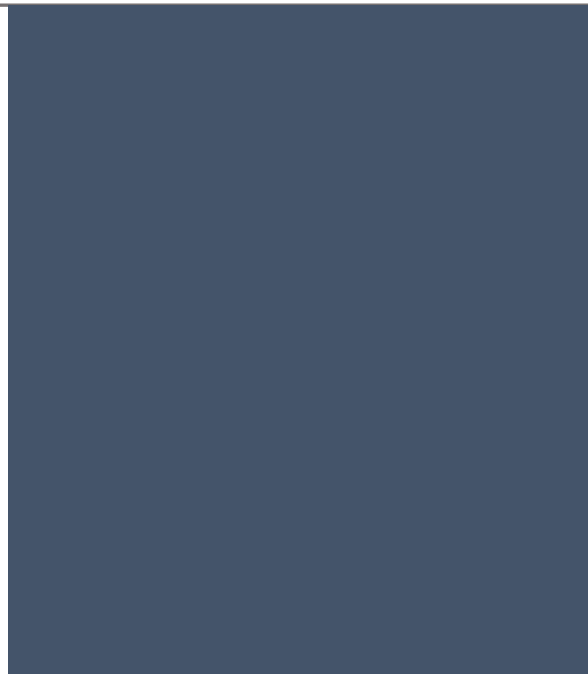




Emotion Recognition

Esame Fondamenti Web

Conte Gabriele
D'Agostino Pierantonio
Rosangela De Giosa
Leo Daniele
Troccoli Tommaso Gabriele



Contents

SCENARIO APPLICATIVO	2
Obbiettivi	2
Utenti target dell'applicazione	2
ARCHITETTURA DELL'APPLICAZIONE	3
Panoramica Architetturale	3
Tecnologie Utilizzate	4
Principi Architetturali	4
Flusso dell'Applicazione	5
Benefici Architetturali	5
Gestione degli Errori e Sicurezza	6
DIAGRAMMA DEI CASI D'USO	7
DIAGRAMMA DELLE SEQUENZE	9
Fase di Registrazione e Login	9
Fase di Sottomissione delle Risposte	9
MODELLO DEI DATI	11
DOCUMENTAZIONE API	17
Autenticazione e gestione della sessione	17
Gestione delle immagini	18
Registrazione dei risultati emozionali	18
Gestione Errori	18
Integrazione WebSocket	19
COMPONENTI REACT	20
Componenti principali	20
Pagine	21
Custom Hooks	21
Stato globale e flusso dati	23

SCENARIO APPLICATIVO

Emotion Recognition è una web application innovativa, orientata alla ricerca, progettata per analizzare le emozioni umane in risposta a stimoli visivi, attraverso l'integrazione di tecnologie di face analysis, Natural Language Processing (NLP) e una User Experience guidata.

L'app consente agli utenti di creare un profilo personale ed essere guidati in un percorso interattivo. Durante l'esperienza, vengono mostrate una serie di immagini pensate per evocare specifiche emozioni. Per ciascuna immagine, l'utente è invitato a:

- selezionare l'emozione percepita tra quelle disponibili,
- fornire una breve descrizione testuale del proprio stato emotivo,
- consentire la registrazione video tramite webcam per il rilevamento automatico delle espressioni facciali.

L'applicazione utilizza la fotocamera frontale per acquisire in tempo reale le reazioni emotive dell'utente, elaborandole tramite tecniche avanzate di riconoscimento facciale. I dati raccolti – emozioni dichiarate, descrizioni testuali e segnali visivi, vengono successivamente elaborati da un modello di Intelligenza Artificiale, al fine di generare un'analisi emotiva approfondita e personalizzata.

Obbiettivi

L'applicazione è stata progettata con l'obiettivo di supportare attività di ricerca nel campo dello studio delle emozioni, rendendosi adatta all'impiego in contesti accademici, clinici e sperimentali.. Consente la raccolta di dati sia demografici sia emozionali, costruendo così un bacino informativo utile per svolgere analisi statistiche e comportamentali.

Inoltre, l'app mette a disposizione strumenti standardizzati per la valutazione delle emozioni, al fine di garantire coerenza e affidabilità nei risultati ottenuti.

Una funzionalità chiave dell'applicazione è la possibilità di registrare video delle reazioni emotive degli utenti in tempo reale, offrendo così un ulteriore livello di approfondimento qualitativo nell'analisi dei dati.

Utenti target dell'applicazione

L'applicazione è rivolta a un pubblico specifico, composto da utenti che operano, studiano o sono coinvolti in ambiti legati alla psicologia, alle neuroscienze e all'analisi dei dati.

I principali destinatari sono i partecipanti a studi psicologici, che attraverso l'utilizzo della piattaforma contribuiscono attivamente alla raccolta di dati emozionali e comportamentali. Un secondo gruppo target è costituito da ricercatori e studenti impegnati in attività accademiche nei settori della psicologia e delle neuroscienze, per i quali l'app rappresenta uno strumento utile sia per la raccolta sia per l'analisi dei dati.

Anche i data scientist che lavorano su progetti di riconoscimento delle emozioni possono beneficiare delle funzionalità dell'app, soprattutto in termini di disponibilità di dati strutturati e integrazione con strumenti di

elaborazione

automatica.

Il servizio, infine, è anche pensata per tutti gli utenti non specialisti comunque motivati a contribuire attivamente alla ricerca scientifica nel campo delle emozioni.

ARCHITETTURA DELL'APPLICAZIONE

Panoramica Architetture

L'applicazione adotta un'architettura moderna, modulare e scalabile, progettata per offrire un'esperienza utente interattiva e per supportare flussi di lavoro complessi legati all'analisi emozionale.

La piattaforma è organizzata in una struttura multilivello che distingue chiaramente il frontend, il backend e i servizi di infrastruttura.

Il frontend, sviluppato in **React**, gestisce l'interfaccia utente, l'interazione con webcam e microfono, la raccolta delle risposte emotive, e la comunicazione in tempo reale con il backend. Utilizza inoltre componenti modulari organizzati in cartelle specifiche (components/, pages/, hooks/) e si affida a **React Router 6** per la gestione della navigazione tra le diverse schermate. Il design è reso responsivo e mobile-first, grazie all'adozione di **Bootstrap 5** e **Bootstrap Icons** che garantisce una user experience uniforme e accessibile su diversi dispositivi. La gestione delle operazioni di profilazione utente, visualizzazione delle immagini, registrazione delle reazioni emotive sono gestite tramite integrazione **WebRTC** che consente l'accesso alla webcam e la gestione dei video in tempo reale.

Il backend rappresenta il cuore funzionale dell'applicazione e si compone di **due microservizi indipendenti**, sviluppati rispettivamente in **Java (Spring Boot)** e **Python (FastAPI)**, ciascuno con responsabilità specifiche ma strettamente integrate tra loro.

Il **backend Java**, basato su **Spring Boot 3.4.4**, si occupa principalmente della **gestione degli utenti**, dell'**autenticazione e autorizzazione tramite JWT**, della **gestione delle immagini** da presentare all'utente e della **registrazione dei risultati**. L'autenticazione avviene tramite token JWT inviati nelle intestazioni HTTP, garantendo la protezione degli endpoint e il controllo degli accessi. Questo servizio gestisce inoltre la comunicazione tra il frontend e il classificatore video, orchestrando le richieste e archiviando i dati nel database **PostgreSQL** e nello **storage Azure Blob**.

Il **servizio Python**, sviluppato con **FastAPI**, è dedicato all'**analisi emozionale** dei contenuti video tramite **modelli di deep learning** basati su **PyTorch**. Integra una rete neurale convoluzionale personalizzata (**ResEmoteNet**), esegue il **pre-processing dei frame** con **OpenCV**, e restituisce la classificazione delle emozioni con relativi punteggi di confidenza. FastAPI è stato scelto per le sue performance elevate e per il supporto nativo a **OpenAPI**, che consente la generazione automatica della documentazione interattiva. Il servizio è inoltre protetto da **middleware JWT**, assicurando la sicurezza nelle chiamate API.

Entrambi i servizi sono progettati per essere **scalabili, modulari e containerizzati** tramite **Docker**, con configurazione multistage per ottimizzare le immagini. La comunicazione tra i microservizi avviene in modo sicuro, con attenzione alla **validazione degli input**, alla **gestione centralizzata degli errori** e al rispetto delle **best practice di sicurezza**.

Infine, la terminazione delle connessioni sicure (SSL/TLS) e il **bilanciamento del carico** sono affidati a un **reverse proxy Nginx**, che funge da punto di ingresso centralizzato per l'intero sistema.

La persistenza dei dati è affidata a **Azure SQL Server**, un sistema di gestione relazionale scalabile e sicuro, che consente di memorizzare in modo strutturato i dati anagrafici degli utenti, le risposte emozionali, le informazioni sulle sessioni sperimentali e i risultati dell'analisi automatica. Il database è integrato con il backend attraverso meccanismi di connessione sicuri, con supporto a query parametrizzate, vincoli di integrità referenziale e tracciamento delle operazioni.

Tecnologie Utilizzate

Frontend

- **Linguaggi e librerie:** React 19.1.0, React Router 6.26.2, React Bootstrap 2.10.9
- **Stilizzazione:** Bootstrap 5.3.5, Bootstrap Icons 1.13.1
- **Media/Web APIs:** WebRTC, MediaRecorder API, Canvas API, WebSocket API
- **Testing:** Jest, React Testing Library

Backend

- **Linguaggi:** Java (Spring Boot 3.4.4) e Python (FastAPI)
- **Autenticazione:** JWT tramite header HTTP, protezione delle route con Spring Security e middleware FastAPI e OAuth 2.0 di Google
- **Servizi:** Classificazione emozionale da immagini/video tramite AI (ResEmoteNet), gestione immagini e registrazione risultati
- **Comunicazione real-time:** WebSocket server (con gestione stato, utenti e messaggi)

Infrastruttura

- **Containerizzazione:** Docker (build multi-stage per backend Java e FastAPI Python)
- **Web server:** Nginx come reverse proxy, configurato per HTTPS e instradamento delle richieste tra i servizi
- **Sicurezza:** Certificati SSL/TLS tramite OpenSSL, protezione JWT, CORS, input validation e rate limiting

DataBase

- **Database relazionale:** PostgreSQL, integrato tramite JPA/Hibernate nel backend Java
- **Cloud Storage:** Azure Blob Storage per la gestione sicura dei file multimediali

Principi Architettureali

La progettazione segue una serie di principi architetturali consolidati:

- **Component-Based Design:** tutte le interfacce utente sono progettate come componenti modulari e riutilizzabili.

- **Unidirectional Data Flow:** i dati fluiscono in modo controllato grazie alla gestione dello stato centralizzato tramite Context API.
- **Separation of Concerns:** le funzionalità sono chiaramente suddivise tra UI, logica, e gestione dati.
- **Progressive Enhancement:** le funzionalità avanzate (es. WebRTC, WebSocket) sono opzionali e gestite con fallback per i browser non compatibili.
- **Error Boundaries:** errori di rendering sono intercettati da componenti dedicati per evitare blocchi dell'applicazione.

Flusso dell'Applicazione

Il flusso applicativo è strutturato in tappe sequenziali, ognuna delle quali è gestita da una pagina principale e da componenti dedicati:

1. **HomePage:** primo punto di contatto dell'utente con l'applicazione. Presenta l'obiettivo dell'app.
2. **Profile Page:** responsabile della raccolta di dati demografici e del loro collegamento a un account Google verificato. Garantisce che solo gli utenti autenticati possano procedere.
3. **ImagePage:** presentazione di immagini emozionali, attivazione webcam e raccolta di risposte testuali e visive.
4. **FinalPage:** visualizzazione dei risultati, analisi emozionale e riepilogo.

Ogni fase interagisce con il backend per inviare dati, ricevere previsioni emozionali e aggiornare lo stato applicativo.

Benefici Architettureali

L'architettura della piattaforma è stata pensata per garantire robustezza, modularità e facilità di estensione. In particolare:

- **Scalabilità:** la containerizzazione con Docker e la separazione dei servizi backend permettono una crescita progressiva del sistema, fino a un'eventuale architettura a microservizi.
- **Manutenibilità:** l'organizzazione in directory tematiche (componenti, hooks, servizi, contesto) e l'uso di naming coerente e convenzioni (PascalCase, camelCase) semplificano l'intervento degli sviluppatori.
- **Performance:** uso di React.memo, useCallback, debounce, code splitting e caching per ottimizzare il rendering e la trasmissione dati.
- **Esperienza utente:** UI responsiva, gestione avanzata della webcam, indicatori di stato chiari, accessibilità migliorata.
- **Sicurezza:** JWT in cookie HTTP-only, HTTPS obbligatorio, accesso alla webcam protetto, sanitizzazione input, gestione token automatica.

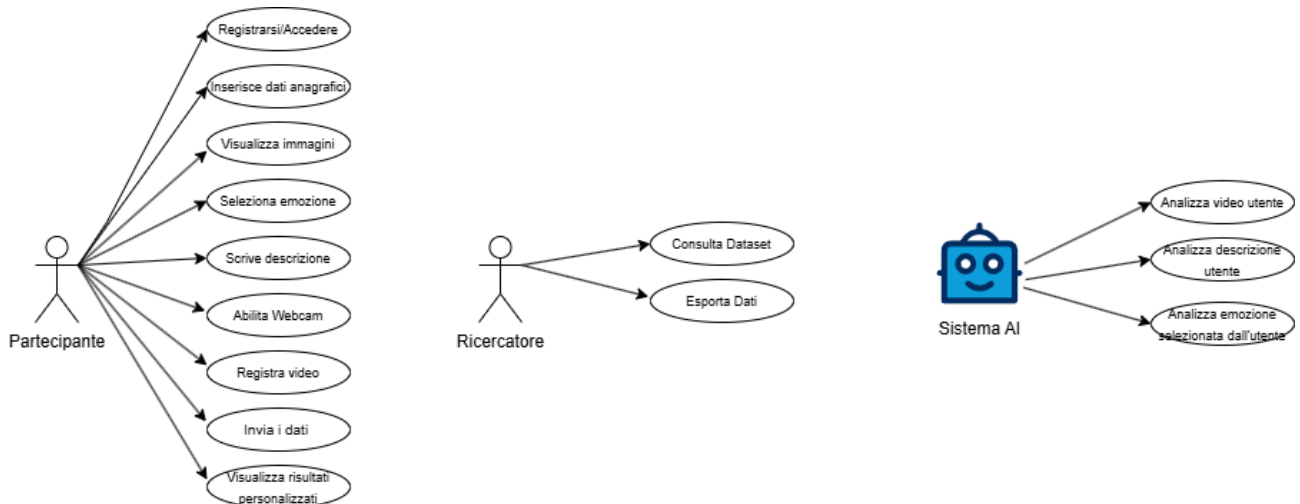
Gestione degli Errori e Sicurezza

La piattaforma implementa una gestione degli errori avanzata, sia a livello di UI sia nelle comunicazioni con i servizi esterni. A livello frontend, i componenti sono avvolti in `ErrorBoundary` per rilevare crash a runtime e sostituirli con fallback UI. Tutte le operazioni di I/O (API REST, WebSocket) sono dotate di sistemi di gestione delle eccezioni e timeout configurabili.

La sicurezza è affrontata in modo multilivello:

- L'autenticazione è basata su **JWT memorizzati in cookie HTTP-only**, evitando vulnerabilità XSS.
- Le rotte sensibili sono protette da componenti `ProtectedRoute`.
- Le registrazioni webcam sono gestite localmente e inviate al backend solo con consenso esplicito.
- Tutte le comunicazioni sono cifrate tramite **HTTPS (SSL/TLS)**, e il backend utilizza certificati generati automaticamente tramite OpenSSL.
- Il sistema WebSocket prevede gestione dello stato, riconnessione automatica con backoff esponenziale, e fallback a modalità offline in caso di disconnessione.

DIAGRAMMA DEI CASI D'USO



Il diagramma dei casi d'uso dell'applicazione Emotion Recognition illustra le principali interazioni tra gli attori esterni e il sistema. Gli attori coinvolti sono:

- **Partecipante:** utente che interagisce con la piattaforma per svolgere l'esperienza emozionale.
- **Ricercatore:** soggetto autorizzato all'accesso e consultazione dei dati raccolti, a fini di studio e analisi.
- **Sistema AI:** componente esterno responsabile dell'elaborazione automatica dei dati video e testuali tramite modelli di Intelligenza Artificiale.

Il partecipante può:

- Registrarsi e autenticarsi nel sistema.
- Inserire le proprie informazioni anagrafiche.
- Visionare una sequenza di immagini emozionali.
- Selezionare l'emozione percepita.
- Fornire una descrizione testuale del proprio stato emotivo.
- Abilitare la webcam e registrare un video durante la visione dell'immagine.
- Inviare tutti i dati raccolti per l'analisi.
- Ricevere e visualizzare i risultati generati.

Il ricercatore può:

- Accedere alla sezione di consultazione dei dati emozionali raccolti.
- Esportare i dataset per attività di analisi, studio e reportistica.

Il sistema AI riceve i dati e svolge due compiti distinti:

- Analizzare i video delle espressioni facciali per rilevare reazioni emotive.

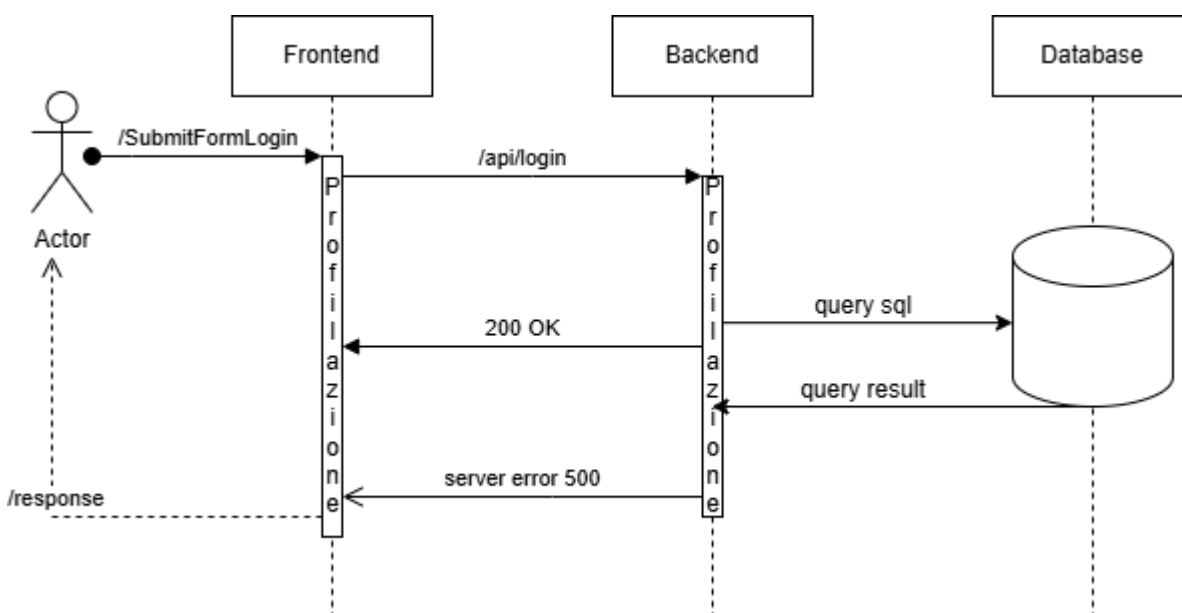
- Eseguire NLP (Natural Language Processing) sul testo fornito dall'utente per identificare l'emozione espressa verbalmente.

DIAGRAMMA DELLE SEQUENZE

Fase di Registrazione e Login

Al primo accesso alla piattaforma, l'utente interagisce con l'interfaccia frontend, compilando un modulo di registrazione contenente le informazioni personali richieste (es. nome, email, password). Questi dati vengono inviati al backend, che provvede a verificarne la validità e, nel caso di un nuovo utente, a registrarli nel database. In caso di esito positivo, il database conferma l'avvenuto salvataggio inviando un messaggio di risposta al backend, il quale a sua volta invia un messaggio di conferma al frontend. L'utente viene così autenticato e reindirizzato alla pagina principale.

In caso di login, le credenziali inserite vengono confrontate con quelle già presenti nel database. Se l'autenticazione ha esito positivo, l'utente accede all'applicazione.



Fase di Sottomissione delle Risposte

Una volta autenticato, l'utente accede alla schermata principale dell'applicazione. Il frontend effettua una richiesta al backend per ottenere un set di immagini salvate nel database. Una volta ricevute, le immagini vengono mostrate all'utente all'interno dell'interfaccia.

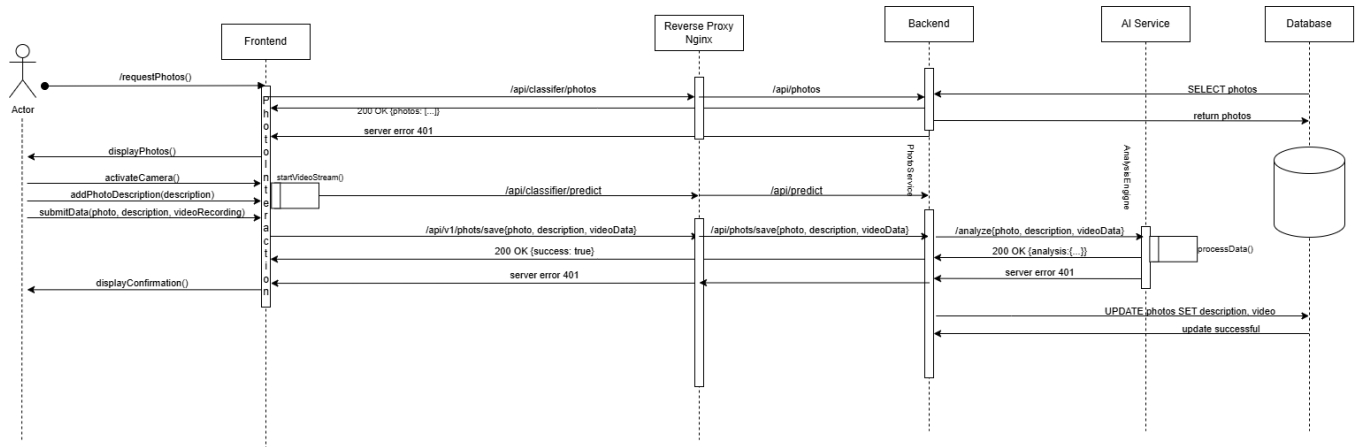
L'utente, per poter procedere, deve autorizzare l'accesso alla webcam. Una volta attivata, viene richiesto all'utente di compilare un modulo in cui inserisce:

- una **descrizione soggettiva** dell'immagine visualizzata,
- l'**emozione percepita** in risposta alla visione dell'immagine,
- e viene nel frattempo acquisita un'immagine del volto tramite webcam.

Questi dati vengono raccolti dal frontend e inviati al backend, il quale li inoltra a un modulo di **servizio di intelligenza artificiale** dedicato all'analisi emotiva. L'AI elabora i dati ricevuti e restituisce al backend una valutazione automatica delle emozioni riconosciute.

Il backend aggrega le informazioni fornite dall'utente e quelle generate dall'AI, quindi procede al salvataggio di tutti i dati nel database. Se il salvataggio va a buon fine, il database invia una risposta di successo al backend, che a sua volta comunica l'esito positivo al frontend.

Il frontend, infine, notifica all'utente l'avvenuta sottomissione dei dati tramite un messaggio di conferma.



MODELLO DEI DATI

Il modello dei dati dell'applicazione è stato progettato per supportare la raccolta, la conservazione e l'analisi strutturata di informazioni relative agli utenti, alle loro reazioni emotive e all'elaborazione automatizzata tramite modelli di Intelligenza Artificiale. L'intero sistema si basa su un database relazionale PostgreSQL, che garantisce coerenza dei dati, integrità referenziale tra le entità (ad esempio tra utenti e reazioni) e la scalabilità necessaria per un ambiente cloud.

L'architettura del modello segue una logica a livelli, in cui le entità principali (Entity) corrispondono alle tabelle del database, mentre i livelli intermedi (DTO, servizi, controller) mediano tra i dati e il frontend.

UserEntity e UserRepository

Il sistema gestisce gli utenti tramite la classe UserEntity, mappata sul database per contenere informazioni personali. I dati vengono manipolati dal UserRepository, responsabile delle operazioni CRUD sull'entità utente.

Campi principali:

- id (PK): identificativo numerico
- email: univoca, usata per il login
- googleId: Id Google login (unique, not null)
- emailVerify: campo boolean
- nickname, age, gender, nationality: dati anagrafici utili per l'analisi

UserService

Classe di servizio che gestisce le funzionalità di autenticazione e interazione utente:

- login(UserEntity, HttpServletResponse): autentica l'utente e restituisce JWT
- logout(HttpServletRequest, HttpServletResponse): revoca i token inserendoli nella blacklist
- packJwts(String user, long expiration): crea access/refresh tokens firmati

Il servizio è fortemente integrato con:

- JwtProvider: per la generazione e verifica dei token
- BlacklistTokenRepository: per invalidare i token al logout
- AzureStorageService: per scaricare immagini da Azure Blob

AzureStorageService

Questo servizio gestisce il caricamento dinamico delle immagini da presentare all'utente:

- getRandomBlobNames(int): seleziona immagini random

- `downloadImage(String)`: recupera un'immagine specifica come array di byte
- `listAllBlobs()`: mostra tutte le immagini disponibili (admin-side)

Le immagini sono archiviate in Azure Blob Storage e accessibili solo attraverso richieste autorizzate, garantendo sicurezza e scalabilità.

ReactionsEntity

La classe `ReactionsEntity` rappresenta la risposta emozionale di un utente a un'immagine. Ogni istanza corrisponde a una singola reazione all'interno di una sessione.

Attributi principali:

- `id`: identificatore univoco della reazione (PK)
- `userId`: riferimento all'utente (FK verso `UserEntity`)
- `image`: identificatore o path dell'immagine mostrata
- `imageReaction`: emozione selezionata dall'utente
- `imageDescription`: descrizione testuale della reazione
- `aiComment`: valutazione generata dal sistema AI

Relazioni:

- Relazione molti-a-uno (N:1) con `UserEntity`: ogni utente può avere più reazioni
- Aggregata tramite il DTO `ResultPayload` in fase di salvataggio

ResultPayload e flusso dati

Le reazioni vengono ricevute dal backend attraverso un payload aggregato (`ResultPayload`), che contiene una lista di `ImageDescriptionAndReactionPayload`, ciascuno dei quali rappresenta una risposta a una singola immagine.

Il metodo `registerResult(ResultPayload)` nel `ReactionsService` elabora questo oggetto, crea le entità `ReactionsEntity` corrispondenti e le salva nel database tramite `ReactionsRepository`.

ReactionsService

Responsabile della logica di business legata alla registrazione delle risposte emozionali. Gestisce:

- La validazione dell'utente (`userId` esistente)
- La trasformazione dei payload in entità persistenti
- Il salvataggio batch delle reazioni

Testato tramite la classe `ReactionsServiceTest`, che verifica:

- Registrazione avvenuta con successo (testRegisterResult_Success)
- Errori nel caso di utenti non trovati (testRegisterResult_UserNotFound)

ReactionEnum

L'emozione selezionata è gestita come un enumeratore (ReactionEnum), che garantisce:

- Consistenza nei valori disponibili (es. joy, sadness, surprise, fear...)
- Facile mappatura tra frontend e backend
- Validazione dei dati in ingresso tramite valueOf(String)

LoginResponse

Quando un utente effettua il login, il sistema restituisce un oggetto LoginResponse che include:

- userId: identificativo dell'utente autenticato
- imagesName: lista delle immagini da somministrare (generate da AzureStorageService)

Componenti dati per autenticazione e sicurezza

L'autenticazione degli utenti avviene tramite JWT (JSON Web Token) e si basa su una serie di componenti modulari:

JwtProvider

Classe centrale per la generazione, validazione e decodifica dei token JWT. Espone metodi per:

- Estrarre username, scadenza e claims dal token
- Validare e generare token a partire da dati utente
- Verificare l'integrità e la validità temporale del token

JwtAuthFilter

Filtro che intercetta le richieste HTTP in ingresso e applica i controlli di autenticazione:

- Verifica se il token è presente e valido
- In caso di errori, invoca build401Error() per segnalare l'accesso non autorizzato

SecurityConfigurer

Configura la catena dei filtri Spring Security. Gestisce:

- La definizione delle rotte pubbliche e protette
- Le regole CORS e la protezione CSRF

- L'inserimento del filtro `JwtAuthFilter` nella pipeline

`JwtProviderTest`, `JwtAuthFilterTest`, `SecurityConfigurerTest`

Test unitari che validano il corretto funzionamento delle componenti di sicurezza in scenari come:

- Token scaduti, mancanti, errati
- Accesso a risorse pubbliche o protette
- Composizione della catena di sicurezza

Payload dei dati emozionali

Durante la raccolta e restituzione dei dati emozionali, il sistema utilizza oggetti DTO (Data Transfer Object) progettati per la trasmissione via API REST.

`ImageDescriptionAndReactionPayload`

Contiene una singola reazione a una specifica immagine:

- `image`: identificatore o URL dell'immagine
- `description`: testo fornito dall'utente sull'esperienza emotiva
- `reaction`: emozione selezionata
- `aiComment`: valutazione automatica prodotta dal modello AI

`ResultPayload`

Contiene una lista di tutte le risposte fornite durante una sessione:

- `imagesDescriptionsAndReactions`: lista di `ImageDescriptionAndReactionPayload`
- Viene restituito alla fine della sessione per mostrare il riepilogo personalizzato all'utente

Gestione della Blacklist dei Token JWT

Per rafforzare la sicurezza del sistema di autenticazione, l'applicazione include un meccanismo di invalidazione dei token tramite blacklist. Questo consente di disattivare i token JWT prima della loro scadenza naturale, ad esempio in caso di logout, cambio credenziali o sospetto di uso improprio.

`BlacklistTokenEntity`

Entità persistente che rappresenta un token invalidato. Contiene:

- `id`: chiave primaria
- `token`: stringa del JWT da invalidare
- `expirationDate`: scadenza pianificata del token

`BlacklistTokenRepository`

Componente di accesso ai dati (repository) per la lettura e scrittura dei token revocati.

BlacklistJwtFilter

Filtro integrato nella catena di sicurezza che intercetta le richieste in ingresso e:

- Estrae il JWT dal cookie
- Verifica la presenza del token nella blacklist
- Blocca l'accesso se il token è revocato
- Passa il controllo ai filtri successivi in caso contrario

BlacklistJwtFilterTest

Test di unità che coprono casi specifici come:

- Token validi non blacklistati
- Token esplicitamente invalidati
- Richieste senza cookie o token
- Accesso a rotte pubbliche e di login
- Gestione utenti, accesso e immagini

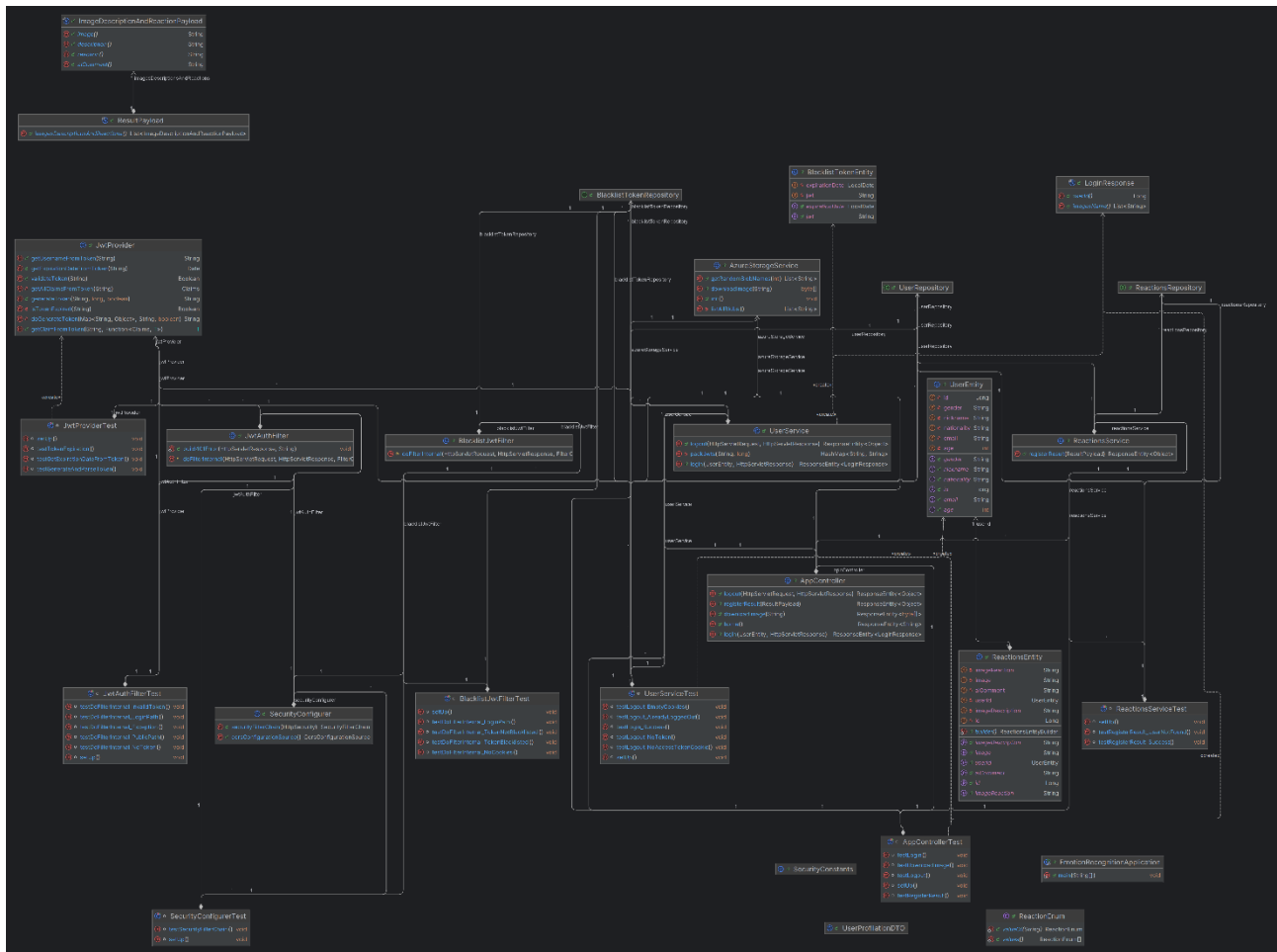
AppController (REST API controller)

Il controller espone i principali endpoint HTTP:

- POST /google-login: login dell'utente → genera i JWT
- POST /logout: logout dell'utente → inserisce token in blacklist
- POST /registerResult: salva le reazioni emozionali
- GET /downloadImage: recupera un'immagine da Azure
- GET /home: ritorna dati di benvenuto o configurazione iniziale

AppControllerTest, UserServiceTest

I test automatizzati convalidano il comportamento degli endpoint e delle logiche interne con scenari multipli (token valido, scaduto, assente, immagini non trovate, login già effettuato, ecc.).



DOCUMENTAZIONE API

Il backend dell'applicazione espone una serie di API RESTful e WebSocket, progettate per supportare in modo sicuro ed efficiente le operazioni di autenticazione, raccolta dati emozionali e analisi automatica.

Autenticazione e gestione della sessione

Il sistema di autenticazione è basato su **JWT (JSON Web Token)** ed è progettato per garantire riservatezza, integrità dei dati e protezione contro gli attacchi più comuni (come CSRF e XSS).

L'accesso avviene tramite l'endpoint `POST /api/login`, al quale l'utente invia le proprie informazioni personali (nickname, email, età, genere, nazionalità). In caso di successo, il server genera un access token, valido per 60 minuti, e un refresh token, con una validità di 120 minuti. Entrambi i token vengono memorizzati in cookie HTTP-only, protetti con attributi Secure e SameSite=Strict, per impedire l'accesso da parte di script maligni e ridurre il rischio di attacchi cross-site.

Questa strategia di autenticazione consente una gestione sicura e trasparente delle sessioni, in cui l'access token viene rinnovato automaticamente finché il refresh token resta valido. L'endpoint `POST /api/logout` consente invece la disconnessione dell'utente, invalidando i token e cancellando i cookie dal browser.

Durante il login, oltre all'autenticazione, il sistema restituisce all'utente un array contenente 10 immagini selezionate casualmente da Azure Blob Storage, che saranno successivamente mostrate durante la sessione.

Endpoint Autenticazione

Per semplificare l'accesso degli utenti e migliorare la sicurezza dell'applicazione, è stato integrato il sistema di autenticazione federata offerto da Google. Questo endpoint consente l'autenticazione tramite Google Sign-In, delegando la verifica dell'identità dell'utente a un provider esterno affidabile.

POST /api/v1/google-login

Questo endpoint consente l'autenticazione degli utenti tramite Google Sign-In. La richiesta deve contenere nel corpo, il token JWT ottenuto da Google e l'insieme dei dati del profilo utente raccolti nel frontend: email, Google ID, nickname, età, genere e nazionalità.

Il backend successivamente verifica la validità del token JWT utilizzando le chiavi pubbliche fornite da Google. Una volta verificato il token e confermata l'autenticità dell'email (controllando il campo `email_verified`), l'API provvede a creare o aggiornare l'utente nel database utilizzando il Google ID come identificativo univoco. Successivamente, vengono generati i token JWT utilizzati internamente per l'autenticazione dell'utente, e viene restituita la risposta contenente tali token, l'ID interno dell'utente e l'elenco di immagini da Azure Blob Storage.

Questo flusso elimina la necessità di gestire credenziali locali, sfruttando direttamente l'autenticazione federata offerta da Google, offrendo così un'esperienza di login semplificata e più sicura.

POST /api/logout

Tramite questo endpoint è possibile terminare la sessione dell'utente. La richiesta deve includere il cookie con l'access token ed in caso di successo, il server risponde con un messaggio di conferma, provvedendo ad invalidare il token, cancellare i cookie JWT e aggiungere il token alla blacklist.

Gestione delle immagini

Le immagini da analizzare sono ospitate su **Azure Blob Storage**.

GET /api/download-image

Tramite questo endpoint è possibile scaricare un'immagine salvata su Azure Blob Storage, da utilizzare per la visualizzazione all'utente. Il nome del file da scaricare deve essere specificato come parametro nella query string. L'accesso è consentito esclusivamente agli utenti autenticati.

A fronte di una richiesta valida e di un'immagine presente nello storage, il server restituisce il contenuto binario del file accompagnato da intestazioni HTTP appropriate. L'endpoint prevede la validazione del parametro e gestisce correttamente eventuali errori, restituendo risposte esplicite in caso di token non valido, parametro mancante, file inesistente o errori di comunicazione con Azure Storage.

Registrazione dei risultati emozionali

Una delle funzionalità principali dell'applicazione consiste nella raccolta e registrazione delle reazioni emotive degli utenti, generate in risposta alle immagini mostrate. Questi dati vengono inviati tramite **POST /api/register-result**, con un payload che include un array di oggetti, ciascuno contenente:

- il nome dell'immagine visualizzata;
- una breve descrizione soggettiva fornita dall'utente;
- l'emozione percepita (es. joy, sadness, fear...);
- un commento AI-generato basato sull'analisi della descrizione.

I dati inviati vengono salvati nel database PostgreSQL e associati al profilo dell'utente autenticato, per essere successivamente utilizzati a fini di analisi o visualizzazione. In caso di esito positivo, ovvero di salvataggio dati andato a buon fine, il sistema restituisce un messaggio di conferma insieme al codice 201 Created. Le reazioni vengono registrate nella tabella `user_reactions` e collegate all'utente corrispondente.

Eventuali errori, come input non valido o utente inesistente, vengono gestiti con risposte esplicite e messaggi dedicati.

Gestione Errori

Tutti gli endpoint restituiscono risposte di errore standardizzate, utili sia per il frontend che per eventuali strumenti di debug (come Postman o curl). Tra gli errori previsti:

- 400 Bad Request: per input malformati o campi mancanti;
- 401 Unauthorized: in assenza o invalidità del token JWT;
- 404 Not Found: in caso di risorse inesistenti (es. immagine non trovata);
- 500 Internal Server Error: per eccezioni lato server non gestite.

Il frontend è progettato per intercettare automaticamente gli errori, in particolare i 401, attivando il flusso di rinnovo del token. Se anche il refresh fallisce, il sistema rimuove i cookie e reindirizza l'utente alla homepage, garantendo una gestione coerente delle sessioni.

Integrazione WebSocket

L'applicazione sfrutta la tecnologia **WebSocket** per abilitare funzionalità in tempo reale, migliorando l'interattività e la sincronizzazione tra client e server.

- **URL configurabile:** il WebSocket endpoint è specificato tramite la variabile d'ambiente `REACT_APP_WS_URL`.
- **Messaggi gestiti:**
 - **Aggiornamento utenti attivi** (conteggio visualizzato nella AppNavbar)
 - **Analisi in tempo reale delle emozioni** durante l'input testuale dell'utente
 - **Notifiche** relative a connessioni/disconnessioni di altri utenti

Il client WebSocket, implementato nel custom hook `useWebSocket`, offre:

- **Riconnessione automatica:** con strategia di **backoff esponenziale** fino a un numero massimo di tentativi.
- **Gestione avanzata dei messaggi:** parsing centralizzato con dispatch basato su tipo di messaggio (`message.type`).
- **Pulizia delle risorse:** disconnessione sicura e rimozione degli event listener al cambio pagina o disconnessione dell'utente (prevenzione di memory leak).

COMPONENTI REACT

Il frontend si basa su **React 19.1.0** con un'architettura a componenti, gestione dello stato con Context API e hooks personalizzati per estendere funzionalità di webcam, autenticazione e comunicazione real-time.

Componenti principali

Tutti i componenti principali sono organizzati nella cartella `components/` e sono importabili in modo centralizzato tramite il file `index.js`.

- **AppNavbar.js**
Componente responsabile della barra di navigazione principale dell'applicazione. Include la visualizzazione dinamica del numero di utenti attualmente connessi, grazie all'integrazione con il sistema WebSocket. È progettato per essere completamente responsivo, adattandosi a diverse dimensioni di schermo.
- **WebcamDisplay.js**
Componente dedicato alla gestione della webcam, che fornisce una preview video in tempo reale. Include il supporto per la cattura di frame e l'interazione con gli hook personalizzati per la gestione dello stream video. È un elemento chiave nella fase centrale del flusso emozionale.
- **LoadingSpinner.js**
Componente visuale che mostra uno spinner di caricamento uniforme, utilizzato in tutte le pagine durante operazioni asincrone o transizioni di stato. Contribuisce a migliorare la percezione di reattività dell'interfaccia utente.
- **ErrorBoundary.js**
Componente React che agisce da "guardiano" per gli errori di rendering. Avvolge i sotto-componenti critici dell'applicazione e, in caso di crash, mostra un messaggio di fallback evitando che l'intera interfaccia si blocchi. Questo meccanismo aumenta la stabilità e l'affidabilità percepita del sistema.
- **GoogleSignIn.js**
Componente React che integra la logica del login con Google antecedente alla registrazione dell'utente.
- **NotificationSystem.js**
Componente React utile per inniettare banner temporanei di errore nel DOM.
- **ProtectedRoute.js**
Componente React wrapper che protegge l'accesso alle pagine che richiedono autenticazione (ImagePage e FinalPage).
- **index.js**
File centrale che esporta tutti i componenti presenti nella cartella, facilitando importazioni coerenti e ordinate nei moduli superiori dell'applicazione.

Pagine

L'interfaccia utente dell'applicazione è organizzata in un flusso sequenziale di **quattro pagine principali**, ciascuna delle quali corrisponde a una fase specifica dell'esperienza utente. La navigazione è gestita tramite **React Router**, con rotte ben definite che guidano l'utente attraverso l'intero processo, dalla registrazione fino alla visualizzazione dei risultati.

- **HomePage** (/)
È la schermata iniziale dell'applicazione. Ha la funzione di accogliere l'utente, fornirgli le istruzioni generali sull'utilizzo della piattaforma e introdurre il contesto della sessione emozionale. In questa fase l'utente può iniziare il percorso guidato accedendo al modulo di registrazione.
- **ProfilePage** (/profile)
In questa pagina viene presentato un modulo per la raccolta dei dati anagrafici dell'utente. Le informazioni richieste (come età, genere, nazionalità, nickname, email) vengono memorizzate nel contesto applicativo e inviate al backend per l'autenticazione e l'associazione con la sessione in corso. L'inserimento dei dati è obbligatorio per poter accedere alla fase sperimentale.
- **ImagePage** (/image)
Si tratta della fase centrale dell'applicazione, in cui viene somministrata una sequenza di immagini progettate per evocare specifiche emozioni. Durante la visualizzazione, l'utente è invitato a selezionare l'emozione percepita, a descrivere il proprio stato emotivo tramite un breve testo e, se autorizzato, a registrare il video della propria espressione facciale in tempo reale tramite webcam. Tutti i dati raccolti sono poi inviati al backend per l'analisi.
- **FinalPage** (/final)
Rappresenta la fase conclusiva del percorso. Qui vengono mostrati i risultati personalizzati generati dal sistema AI, basati sull'analisi delle espressioni facciali e delle descrizioni testuali fornite durante la sessione. L'utente può visualizzare un riepilogo delle emozioni rilevate e, se previsto, ricevere un feedback qualitativo sull'andamento della sessione.

Custom Hooks

L'applicazione fa uso di una serie di **custom React hooks** definiti all'interno della directory `hooks/`, con lo scopo di incapsulare e riutilizzare logiche complesse in maniera modulare e leggibile. Questi hook personalizzati rappresentano un'estensione coerente dell'architettura basata su componenti, migliorando la manutenzione del codice e favorendo la separazione delle responsabilità.

- **useWebSocket.js**
Gestisce la connessione WebSocket con il backend in tempo reale. Include la logica per la connessione e disconnessione automatica, la ricezione e l'invio di messaggi, il monitoraggio dello stato del canale, e l'implementazione di strategie di riconnessione progressiva (es. backoff esponenziale). Espone lo stato della connessione, il conteggio degli utenti attivi e metodi per interagire dinamicamente con il server.
- **useAuth.js**
Gestisce lo stato di autenticazione dell'utente, verificando la presenza dei token JWT, controllando i cookie HTTP-only e aggiornando lo stato globale in base alla validità del token. Viene utilizzato da componenti come `ProtectedRoute` per garantire l'accesso controllato alle pagine protette.

- **useWebcam.js**

Incapsula l'intera logica legata alla webcam del dispositivo: attivazione, gestione del flusso video, cattura di frame o blob video, e controllo dell'interfaccia utente. È progettato per gestire errori comuni (come mancanza di permessi o dispositivi) e integra i vincoli mediatici consigliati (es. risoluzione, codec, bitrate).

Stato globale e flusso dati

La gestione dello stato globale all'interno dell'applicazione *Emotion Recognition* è centralizzata attraverso un **contesto applicativo personalizzato**, definito nel file `AppContext.js`, all'interno della directory `context/`. Questa struttura utilizza le API native di React (`createContext`, `useReducer`, `useMemo`) per mantenere lo stato condiviso tra i componenti in modo efficiente, reattivo e modulare.

Il cuore del sistema è un **reducer** che definisce lo stato iniziale e gestisce una serie di azioni (action types) specifiche per ogni tipo di aggiornamento. L'inizializzazione dello stato prevede diverse sezioni funzionali, tra cui:

- **profile**: contiene i dati identificativi e demografici dell'utente (nickname, email, età, genere, nazionalità, userId)
- **images**: array delle immagini da presentare all'utente, ottenute al login
- **currentImageIndex**: indice corrente per la sequenza di immagini
- **imageReactions**: raccolta delle risposte emotive generate per ciascuna immagine
- **assessmentResults**: risultati analitici finali ottenuti dal sistema AI
- **isLoading** e **error**: gestione degli stati di caricamento ed eventuali errori
- **isAuthenticated**: flag che definisce lo stato di autenticazione dell'utente

L'aggiornamento dello stato avviene esclusivamente tramite dispatch di azioni ben definite, come `SET_PROFILE`, `ADD_IMAGE_REACTION`, `SET_ASSESSMENT_RESULTS`, e così via, garantendo un flusso di controllo chiaro e prevedibile.

Un ulteriore elemento distintivo è l'integrazione con il WebSocket, inizializzato all'interno del contesto tramite il custom hook `useWebSocket`. Questo permette ai componenti figli di accedere alla connessione real-time in modo centralizzato, mantenendo separata la logica di rete dalla presentazione.

Per prevenire render non necessari, l'intero valore del contesto è **memorizzato con `useMemo`**, includendo sia lo stato che i metodi di aggiornamento. Inoltre, viene fornito un custom hook `useAppContext` che semplifica l'accesso al contesto all'interno dei componenti, con validazione del contesto attivo per evitare errori strutturali.

Il **flusso dei dati** segue un paradigma unidirezionale ben definito:

1. L'utente interagisce con l'interfaccia fornendo input (form, selezioni, registrazioni)
2. I dati raccolti aggiornano lo stato globale tramite dispatch
3. Il frontend invia i dati al backend attraverso le API REST
4. Le risposte del server aggiornano nuovamente lo stato (es. risultati AI)
5. Il risultato finale viene presentato all'utente